ENTERPRISE ENGINEERING GUIDE

Ubuntu 24.04 LTS VDI Golden Template Implementation

Building a Production-Ready Linux VDI Golden Image on Proxmox VE with TigerVNC and Apache Guacamole Browser Access

Proxmox Host:	prox000innspark (10.31.1.163)
Guacamole Gateway:	https://10.31.4.55/
Base OS:	Ubuntu Server 24.04 LTS (Noble)
Desktop Environment:	XFCE 4 (Minimal, Lightweight)
Remote Display:	TigerVNC (Port 5901, Display :1)
Template ID:	9051 (ubuntu-2404-vdi-vnc-template-9051)
Validated Clone:	VM 201 (vdi-user-test)
Storage Pool:	local-lvm (LVM-Thin, Discard/TRIM)
Execution Status:	**Verified & Tested OK**

1. Overview and Architecture

Virtual Desktop Infrastructure (VDI) running on Linux provides high-density, cost-effective, and secure virtual workstations. Instead of deploying a resource-intensive desktop distribution, this architecture utilizes **Ubuntu Server 24.04 LTS** as the minimal operating-system base.

On top of this clean baseline, **XFCE 4** provides an enterprise, lightweight graphical interface (~400 MB RAM per idle session), while **TigerVNC** exposes the X11 display on TCP port `5901`. End users access the virtual desktop directly via their web browser

through an enterprise **Apache Guacamole Gateway (10.31.4.55)**.

Layer	Component	Configuration Specification
Hypervisor	Proxmox VE 8.x	Node: `prox000innspark` (10.31.1.163), Kernel 6.8+
Virtual Hardware	QEMU / KVM	Machine: `q35`, Firmware: `OVMF (UEFI)`, 4 vCPUs (host), 4GB RAM
Storage Controller	VirtIO SCSI Single	32 GB SCSI Disk on `local-lvm`, Discard/TRIM enabled, SSD emulation
Networking	VirtIO Paravirtualized	Bridged to `vmbr0`, DHCP IPv4 auto-addressing
Guest Operating System	Ubuntu Server 24.04 LTS	Minimal cloud image baseline (`noble-server-cloudimg-amd64.img`)
Desktop Interface	XFCE 4	Minimal core (xfce4, xfce4-session, thunar, xfce4-terminal, xfwm4)
Remote Display Transport	TigerVNC Server	Display `:1` on TCP Port `5901`, 1080p 24-bit True Color
Client Access Gateway	Apache Guacamole	`https://10.31.4.55/` (HTML5 clientless browser access)

Key Architecture Advantage: Users require no local client installation (no VPN client, no VNC viewer). They simply navigate to `https://10.31.4.55/` in Chrome, Firefox, or Edge, and receive a secure, responsive Linux virtual desktop inside their web browser.

2. Infrastructure Pre-Flight Fixes

During initial deployment on the Proxmox host, two environmental roadblocks were identified and resolved:

2.1 Corporate DNS Resolution Fix

Issue: The Proxmox host was initially configured with Google DNS (`8.8.8.8`). Corporate

network firewall rules blocked outbound UDP/TCP port 53 to public DNS, resulting in `Temporary failure in name resolution` when attempting to download base operating system images.

Resolution: The DNS configuration was updated to the corporate internal DNS resolver:

```
# Updated on Proxmox VE (prox000innspark):

dns1: 10.31.4.2

search: local
```

2.2 Storage Selection: Local SSD vs. Read-Only NFS

Issue: Initial VM creation failed with: `TASK ERROR: unable to create VM - mkdir /mnt/pve/NFS-Storage/images: Read-only file system`. The remote NFS export at `10.31.1.161` was mounted in a read-only state.

Resolution: All virtual machine disks, EFI disks, and cloud-init metadata drives were explicitly targeted to **`local-lvm`** (371 GB free local SSD storage). This eliminated network dependencies and maximized desktop disk I/O performance.

3. Creating the Base Build Virtual Machine (VM 9051)

The build virtual machine was provisioned directly on the Proxmox host shell using the official, cryptographically verified Ubuntu 24.04 LTS Cloud Image.

Step 1: Download Official Ubuntu 24.04 Cloud Image

```
cd /var/lib/vz/template/iso

wget -nc https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img
```

Step 2: Initialize VM 9051 with Modern Virtual Hardware

```
qm create 9051 \

  --name "ubuntu-2404-vdi-gold-build" \
```

```
  --machine q35 \

  --bios ovmf \

  --cpu host \

  --cores 4 \

  --memory 4096 \

  --balloon 0 \

  --scsihw virtio-scsi-single \

  --net0 virtio,bridge=vmbr0 \

  --agent enabled=1 \

  --ostype l26
```

Step 3: Allocate EFI Disk, Import Cloud Image, and Resize

```
# 1. Allocate 4MB OVMF EFI Disk on local-lvm

qm set 9051 --efidisk0 local-lvm:0,pre-enrolled-keys=1,efitype=4m



# 2. Import cloud image into local-lvm

qm importdisk 9051 /var/lib/vz/template/iso/noble-server-cloudimg-amd64.img local-lvm



# 3. Attach imported disk as SCSI 0 with TRIM (discard) and SSD emulation

qm set 9051 --scsi0 local-lvm:vm-9051-disk-1,discard=on,ssd=1,iothread=1



# 4. Resize virtual hard disk to 32 GB

qm disk resize 9051 scsi0 32G
```

Step 4: Configure Cloud-Init Provisioning and Start

```
# Attach Cloud-Init Drive on local-lvm

qm set 9051 --ide2 local-lvm:cloudinit

qm set 9051 --boot order=scsi0



# Configure administrative user and DHCP

qm set 9051 --ciuser vdiadmin

qm set 9051 --cipassword "TempPassword123!"

qm set 9051 --ipconfig0 ip=dhcp



# Power on the VM

qm start 9051
```

Result: VM 9051 booted in UEFI mode, automatically expanded its root filesystem to 32 GB, created the `vdiadmin` user, and acquired IP `10.31.0.232` from DHCP.

4. Operating System and Desktop Environment Setup

Logging into VM 9051 via console or SSH (`ssh vdiadmin@10.31.0.232`), the operating system baseline and desktop packages were installed.

Step 1: System Baseline & QEMU Guest Agent

```
export DEBIAN_FRONTEND=noninteractive

apt-get update && apt-get dist-upgrade -y

apt-get install -y qemu-guest-agent curl wget vim chrony dbus-x11
```

Step 2: Minimal XFCE 4 Desktop Environment

To prevent desktop bloat, heavy consumer suites (LibreOffice, Thunderbird, games) were excluded. The minimal graphical set was installed:

```
apt-get install -y --no-install-recommends \

    xorg x11-xserver-utils xfce4 xfce4-session xfce4-terminal xfce4-appfinder \

    thunar fonts-dejavu-core
```

Step 3: PolicyKit Authorization Fix for Remote Sessions

Under Linux, remote X11/VNC display sessions often trigger intrusive desktop popups requesting administrative credentials for color device profiles (`colord`) or network management. This was permanently resolved by deploying an override rule:

```
mkdir -p /etc/polkit-1/localauthority/50-local.d/



cat <<'EOF' > /etc/polkit-1/localauthority/50-local.d/45-allow-colord.pkla

[Allow Colord and Network for all users]

Identity=unix-user:*

Action=org.freedesktop.color-manager.*;org.freedesktop.NetworkManager.*

ResultAny=yes

ResultInactive=yes

ResultActive=yes

EOF
```

5. TigerVNC Server Setup and Multi-User Service Integration

TigerVNC was chosen for its high rendering speed, low bandwidth utilization, and seamless compatibility with Apache Guacamole.

Step 1: Install TigerVNC

```
apt-get install -y tigervnc-standalone-server tigervnc-common
```

Step 2: Set VNC Password for `vdiadmin`

```
su - vdiadmin -c '

mkdir -p ~/.vnc

echo "TempPassword123!" | vncpasswd -f > ~/.vnc/passwd

chmod 600 ~/.vnc/passwd

'
```

Step 3: Configure XFCE Session Startup Script

Configured `/home/vdiadmin/.vnc/xstartup` to cleanly initialize environment variables and launch XFCE:

```
cat <<'EOF' > /home/vdiadmin/.vnc/xstartup

#!/bin/sh

unset SESSION_MANAGER

unset DBUS_SESSION_BUS_ADDRESS

export XDG_CURRENT_DESKTOP=XFCE

export XDG_SESSION_DESKTOP=xfce

exec startxfce4
```

```
EOF



chmod +x /home/vdiadmin/.vnc/xstartup

chown -R vdiadmin:vdiadmin /home/vdiadmin/.vnc
```

Step 4: Multi-User Mapping and Global Defaults

In Ubuntu 24.04 LTS, TigerVNC manages sessions via native systemd templates. Display `:1` was mapped to `vdiadmin`:

```
# 1. Map Display :1 to user vdiadmin

echo ':1=vdiadmin' > /etc/tigervnc/vncserver.users



# 2. Configure system defaults in Perl format (1080p, remote access allowed)

cat <<'EOF' > /etc/tigervnc/vncserver-config-defaults

$session = "xfce";

$geometry = "1920x1080";

$localhost = "no";

1;

EOF
```

Step 5: Enable and Start Native Systemd Service

```
systemctl daemon-reload

systemctl enable --now tigervncserver@:1.service

systemctl enable --now qemu-guest-agent
```

Verification: Running `ss -tlpn | grep 5901` confirmed: `LISTEN 0 5 0.0.0.0:5901 (Xtigervnc)`. The desktop was verified live via VNC viewer.

6. Generalization, Sealing, and Template Conversion

Before a virtual machine can become a production template, all machine-specific identifiers, temporary session files, package caches, and network leases must be purged. This guarantees that clones do not conflict on IP addresses or cryptographic identities.

Step 1: Execute Sealing Operations Inside the VM

Executed inside VM 9051 as root:

```
# 1. Stop VNC service and clean session logs

systemctl stop tigervncserver@:1.service

rm -f /home/vdiadmin/.vnc/*.log /home/vdiadmin/.vnc/*.pid



# 2. Populate skeleton directory so new users automatically receive VNC xstartup

mkdir -p /etc/skel/.vnc

cp /home/vdiadmin/.vnc/xstartup /etc/skel/.vnc/xstartup

chmod +x /etc/skel/.vnc/xstartup



# 3. Clean APT package caches and residual deb files

export DEBIAN_FRONTEND=noninteractive

apt-get autoremove -y --purge

apt-get clean

rm -rf /var/lib/apt/lists/*

rm -rf /var/cache/apt/archives/*.deb /var/cache/apt/archives/partial/*
```

```
# 4. Clean cloud-init artifacts (enables cloud-init execution on new clones)
cloud-init clean --logs --seed
rm -rf /var/lib/cloud/instances/*
rm -rf /var/lib/cloud/instance
rm -rf /var/lib/cloud/data/*

# 5. Reset machine-id (Guarantees unique DHCP IP lease per clone)
truncate -s 0 /etc/machine-id
rm -f /var/lib/dbus/machine-id
ln -sf /etc/machine-id /var/lib/dbus/machine-id

# 6. Purge SSH host keys (Clones will regenerate unique key pairs)
rm -f /etc/ssh/ssh_host_*_key /etc/ssh/ssh_host_*_key.pub

# 7. Clean temporary files and caches
find /tmp -mindepth 1 -maxdepth 1 -exec rm -rf {} + 2>/dev/null || true
find /var/tmp -mindepth 1 -maxdepth 1 -exec rm -rf {} + 2>/dev/null || true
rm -rf /root/.cache/* /home/vdiadmin/.cache/* 2>/dev/null || true

# 8. Clear bash history, flush filesystem, and power off cleanly
history -c
```

```
sync

shutdown -h now
```

Step 2: Convert VM 9051 to an Immutable Proxmox Template

Once the VM powered off in Proxmox, it was converted to a master template:

```
# On Proxmox Host Shell:

qm set 9051 --name "ubuntu-2404-vdi-vnc-template-9051"

qm template 9051
```

Template Created: VM 9051 was successfully sealed and converted into `ubuntu-2404-vdi-vnc-template-9051`.

7. Apache Guacamole Integration (Gateway: 10.31.4.55)

Your organization operates an **Apache Guacamole Gateway at `https://10.31.4.55/`**. Guacamole provides HTML5 clientless remote desktop connectivity over HTTPS.

Step-by-Step Guacamole Configuration

1. Open a browser and navigate to **`https://10.31.4.55/`**.
2. Log in with administrative credentials.
3. In the top-right navigation dropdown, select **Settings**.
4. Navigate to the **Connections** tab and click **New Connection**.
5. Configure the connection using the exact parameters below:

Section	Field	Configuration Value
Edit Connection	Name	`Ubuntu 24.04 VDI Desktop`
	Location	`ROOT` (or your department group)
	Protocol	**`VNC`**
Network	Hostname	**`<VDI_CLONE_IP>`** (e.g., `10.31.0.232` or new clone IP)

Section	Field	Configuration Value
	Port	**5901**
Authentication	Password	**TempPassword123!** (or user's password)
	Color depth	True color (24-bit)
Display Settings	Cursor	Remote (server-rendered) or Local
	DPI	96 (Standard desktop DPI)

6. Click **Save** at the bottom of the page.
7. Click the **Home** icon in the top left, click the new desktop connection, and your full XFCE Linux desktop will instantly render inside your web browser tab!

Single Sign-On & User Isolation: In production, each user can be assigned their own dedicated clone IP in Guacamole, or Guacamole parameter tokens (`${GUAC_USERNAME}`) can be utilized to automatically route users to their assigned desktops.

8. Creating and Managing VDI Clones

8.1 Deploying a New User Desktop in 15 Seconds

Whenever a new user desktop is needed, deploy a **Full Clone** from the golden template:

Via Proxmox Web GUI:

1. Right-click **ubuntu-2404-vdi-vnc-template-9051** → click **Clone**.
2. Enter Target VM ID (e.g., 202), Name (e.g., vdi-finance-user01).
3. Mode: **Full Clone** | Target Storage: **local-lvm**.
4. Click **Clone**.
5. Click the new VM → click **Start**.
6. Under the **Summary** tab, note the assigned IP address, and add it to Guacamole.

Via Proxmox CLI:

```
# 1. Create full clone
```

```
qm clone 9051 202 --name "vdi-finance-user01" --full 1 --storage local-lvm



# 2. (Optional) Customize user credentials via Cloud-Init

qm set 202 --ciuser "john" --cipassword "SecurePassword2026!"



# 3. Start the clone

qm start 202
```

8.2 Golden Template Maintenance Lifecycle

To apply future security patches or new software, **never edit the template directly**. Follow the immutable versioning lifecycle:

1. Clone the active template to a temporary maintenance VM (e.g., VM `9099`).
2. Start VM `9099`, apply updates (`apt-get dist-upgrade`), and test VNC functionality.
3. Run the generalization/sealing commands from Section 6.
4. Shut down VM `9099` and convert it into a new template version (e.g., `ubuntu-2404-vdi-vnc-template-v2`).
5. Retain the previous template version as a rollback fallback.

Production Sign-Off: All steps outlined in this runbook have been verified and tested live on Proxmox node `prox000innspark` (10.31.1.163) and validated against Apache Guacamole (10.31.4.55). The golden template is production-ready.